ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



BÁO CÁO BÀI TẬP LỚN
TÍCH HỢP VÀ PHÂN TÍCH DỮ LIỆU LỚN

ĐỀ TÀI : Tối ưu và phân tích dữ liệu bán hàng quy mô lớn bằng SQL và các công nghệ Big Data

Giảng viên hướng dẫn : **Bùi Anh Tuấn**

Thành viên Nhóm 12 : **Hà Đức Thắng – 23010284**

Lê Anh Minh – 23010288

Lê Nhật Minh – 23010273

Lớp học phần : **Tích hợp và phân tích dữ liệu lớn -1-1-26(COUR01.LT2)**

Hà Nội , ngày 13 tháng 10 năm 2026

MỤC LỤC

MỤC LỤC 1
PHÂN CHIA CÔNG VIỆC 3
DANH MỤC HÌNH ẢNH 4
1. GIỚI THIỆU ĐỀ TÀI 5
1.1. Lý do chọn đề tài 5
1.2. Mục tiêu và phạm vi thực hiện 5
a. Mục tiêu thực hiện 5
b. Phạm vi thực hiện 6
2. TỔNG QUAN HỆ THỐNG VÀ CÔNG NGHỆ 7
2.1. Kiến trúc tổng thể hệ thống 7
2.2. Công nghệ sử dụng 8
3. XÂY DỰNG CƠ SỞ DỮ LIỆU 13
3.1. Thiết kế các bảng và mối quan hệ 13
3.2. Sinh dữ liệu bán hàng quy mô lớn 15
3.3. Kiểm tra chất lượng và tính toàn vẹn dữ liệu 19
4. TRUY VẤN VÀ TỐI ƯU SQL 22
4.1. Truy vấn SQL nâng cao: JOIN, CTE, Window Functions 22
4.2. Phân tích kế hoạch thực thi bằng EXPLAIN ANALYZE 24
4.3. Tối ưu bằng INDEX và đánh giá hiệu năng 26
5. TÍCH HỢP DỮ LIỆU VÀ DATA WAREHOUSE 28
5.1. Thiết kế Star Schema: Fact và Dimension 28
5.2. Xây dựng quy trình ETL và Staging 30
5.3. Kiểm tra, đối chiếu dữ liệu sau ETL 33
6. PHÂN TÍCH DỮ LIỆU VÀ DASHBOARD 37
6.1. Phân tích dữ liệu bằng Apache Spark 37
6.2. Xây dựng Dashboard bằng Power BI 38
6.3. Phân tích kết quả kinh doanh 39
6.4. Đánh giá chung 41

7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 42
7.1. Kết luận 42
7.2. Hướng phát triển 42
TÀI LIỆU THAM KHẢO 43

PHÂN CHIA CÔNG VIỆC

STT	Tên thành viên	Nhiệm vụ thực hiện	Tỷ lệ công việc
1	Lê Anh Minh	Thiết kế cơ sở dữ liệu PostgreSQL, tạo dữ liệu mẫu, xây dựng quy trình ETL và Data Warehouse.	35%
2	Hà Đức Thắng	Viết và tối ưu SQL, sử dụng EXPLAIN ANALYZE, phân tích dữ liệu bằng PySpark.	35%
3	Lê Nhật Minh	Xây dựng Dashboard Power BI, kiểm thử hệ thống, viết báo cáo và chuẩn bị thuyết trình.	30%
	Tổng cộng		100%

DANH MỤC HÌNH ẢNH

Hình 2.1. Kiến trúc tổng thể của hệ thống. 7

Hình 2.2. Sơ đồ quan hệ dữ liệu nguồn, minh họa các khóa và quan hệ chính. 10

Hình 2.3. Minh họa mô hình Star Schema của Data Warehouse. Các bảng chiều liên kết với bảng fact thông qua khóa ngoại. 11

Hình 3.1. Kết quả kiểm tra số lượng bản ghi trong các bảng dữ liệu nguồn. 15

Hình 3.2. Thống kê số lượng đơn hàng theo trạng thái. 18

Hình 3.3. Tổng doanh thu 20

Hình 4.1. Kết quả EXPLAIN ANALYZE của truy vấn doanh thu theo tháng. 25

Hình 5.1. kiểm tra Data Warehouse 29

Hình 5.2. Truy vấn doanh thu sản phẩm 30

Hình 5.3. Quy trình ETL của hệ thống. 31

Hình 5.4. Số lượng bản ghi 33

Hình 5.5. Đối chiếu doanh thu 34

Hình 5.6. Đối chiếu dữ liệu Staging 35

Hình 6.1. Doanh thu theo năm 38

Hình 6.2. Doanh thu theo danh mục 40

Hình 6.3 Top 10 sản phẩm bán chạy 40

1. GIỚI THIỆU ĐỀ TÀI

1.1. Lý do chọn đề tài

Trong thời đại chuyển đổi số, dữ liệu ngày càng đóng vai trò quan trọng đối với hoạt động quản lý và ra quyết định của doanh nghiệp. Đặc biệt, trong lĩnh vực bán hàng, lượng dữ liệu phát sinh từ khách hàng, sản phẩm, đơn hàng và giao dịch ngày càng lớn. Việc lưu trữ, truy vấn và phân tích khối lượng dữ liệu này một cách hiệu quả là một thách thức đáng quan tâm.

SQL là công cụ quan trọng để quản lý và khai thác dữ liệu trong các hệ quản trị cơ sở dữ liệu quan hệ. Tuy nhiên, khi khối lượng dữ liệu tăng lên, các truy vấn phức tạp có thể mất nhiều thời gian thực thi. Do đó, việc áp dụng các kỹ thuật tối ưu truy vấn SQL, xây dựng quy trình tích hợp dữ liệu và sử dụng công nghệ Big Data để phân tích dữ liệu là cần thiết.

Xuất phát từ thực tế đó, nhóm lựa chọn đề tài "Tối ưu và phân tích dữ liệu bán hàng quy mô lớn bằng SQL và các công nghệ Big Data". Đề tài giúp vận dụng kiến thức về cơ sở dữ liệu, tối ưu SQL, tích hợp dữ liệu, xây dựng Data Warehouse và phân tích dữ liệu bằng PySpark. Đồng thời, việc xây dựng Dashboard giúp trực quan hóa kết quả, hỗ trợ người dùng theo dõi tình hình kinh doanh và đưa ra quyết định phù hợp.

1.2. Mục tiêu và phạm vi thực hiện

a. Mục tiêu thực hiện

Đề tài hướng đến việc xây dựng hệ thống quản lý và phân tích dữ liệu bán hàng quy mô lớn, kết hợp giữa các kỹ thuật xử lý dữ liệu truyền thống và công nghệ Big Data. Các mục tiêu chính bao gồm:

- Xây dựng cơ sở dữ liệu: Thiết kế các bảng lưu trữ thông tin khách hàng, sản phẩm, danh mục, đơn hàng và chi tiết đơn hàng; sử dụng Python để tạo dữ liệu với số lượng lớn phục vụ thực nghiệm.
- Truy vấn và tối ưu SQL: Thực hiện các truy vấn sử dụng JOIN, CTE và Window Functions; phân tích kế hoạch thực thi bằng EXPLAIN ANALYZE; đánh giá hiệu quả của việc sử dụng chỉ mục (INDEX).
- Tích hợp dữ liệu: Xây dựng quy trình ETL để trích xuất, chuyển đổi và nạp dữ liệu từ cơ sở dữ liệu nguồn vào Data Warehouse.
- Xây dựng Data Warehouse: Tổ chức dữ liệu theo mô hình Star Schema, bao gồm bảng sự kiện (Fact) và các bảng chiều (Dimension), nhằm phục vụ truy vấn phân tích.

- Phân tích dữ liệu lớn: Sử dụng PySpark để thực hiện các thao tác xử lý và phân tích dữ liệu.
- Trực quan hóa dữ liệu: Xây dựng Dashboard bằng Power BI để thể hiện các chỉ số như doanh thu theo tháng, doanh thu theo danh mục và các sản phẩm có doanh thu cao.

b. Phạm vi thực hiện

Đề tài tập trung vào dữ liệu bán hàng mô phỏng được lưu trữ trong PostgreSQL, với khoảng 3,11 triệu bản ghi trong các bảng dữ liệu nguồn, bao gồm khách hàng, sản phẩm, danh mục, đơn hàng và chi tiết đơn hàng.

Phạm vi triển khai gồm các nội dung chính:

- Sử dụng PostgreSQL để lưu trữ, quản lý và truy vấn dữ liệu.
- Sử dụng Python để sinh dữ liệu và hỗ trợ quy trình xử lý dữ liệu.
- Sử dụng SQL để khai thác dữ liệu và thực nghiệm các phương pháp tối ưu truy vấn.
- Sử dụng ETL và Data Warehouse để tích hợp, tổ chức dữ liệu phục vụ phân tích.
- Sử dụng PySpark để mở rộng khả năng xử lý và phân tích dữ liệu.
- Sử dụng Power BI để trực quan hóa kết quả phân tích dưới dạng Dashboard.

Đề tài chủ yếu tập trung vào việc thực nghiệm trên dữ liệu bán hàng mô phỏng, đánh giá hiệu năng truy vấn, kiểm tra tính nhất quán dữ liệu sau tích hợp và khai thác các thông tin phục vụ phân tích kinh doanh. Phạm vi đề tài chưa hướng đến việc triển khai một hệ thống bán hàng hoàn chỉnh trong môi trường doanh nghiệp thực tế.

2. TỔNG QUAN HỆ THỐNG VÀ CÔNG NGHỆ

2.1. Kiến trúc tổng thể hệ thống

Hệ thống được xây dựng nhằm phục vụ việc lưu trữ, tích hợp, tối ưu và phân tích dữ liệu bán hàng quy mô lớn. Kiến trúc tổng thể gồm các thành phần chính: sinh dữ liệu, lưu trữ dữ liệu nguồn, truy vấn và tối ưu SQL, tích hợp dữ liệu vào kho dữ liệu, phân tích bằng PySpark và trực quan hóa kết quả bằng Power BI.

Quy trình hoạt động của hệ thống được mô tả như sau:



Hình 2.1. Kiến trúc tổng thể của hệ thống.

Trong kiến trúc này, Python được sử dụng để tạo dữ liệu mô phỏng phục vụ thực nghiệm. PostgreSQL đảm nhiệm việc lưu trữ dữ liệu nguồn và thực hiện các truy vấn SQL. Sau đó, dữ liệu được xử lý thông qua quy trình ETL và đưa vào Data Warehouse để phục vụ các yêu cầu phân tích.

PySpark được sử dụng để mở rộng khả năng xử lý dữ liệu, trong khi Power BI đảm nhiệm việc trực quan hóa các chỉ số kinh doanh thông qua biểu đồ và Dashboard. Các thành phần phối hợp với nhau tạo thành quy trình xử lý dữ liệu từ khâu tạo lập, lưu trữ đến phân tích và trình bày kết quả.

2.2. Công nghệ sử dụng

2.2.1. PostgreSQL

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, hỗ trợ ngôn ngữ SQL và nhiều tính năng phục vụ quản lý dữ liệu. Trong đề tài, PostgreSQL được sử dụng làm hệ quản trị cơ sở dữ liệu chính để lưu trữ dữ liệu bán hàng.

Cơ sở dữ liệu gồm các bảng categories, products, customers, orders và order_items. Các bảng có mối quan hệ với nhau thông qua khóa chính và khóa ngoại, giúp đảm bảo tính toàn vẹn dữ liệu.

Ngoài chức năng lưu trữ, PostgreSQL còn được sử dụng để thực hiện các truy vấn phân tích, kiểm tra kế hoạch thực thi bằng EXPLAIN ANALYZE và đánh giá hiệu quả của việc tạo chỉ mục INDEX. Qua đó, nhóm có thể so sánh thời gian thực thi và phân tích nguyên nhân ảnh hưởng đến hiệu năng truy vấn.

2.2.2. Python

Python là ngôn ngữ lập trình có cú pháp đơn giản, hỗ trợ nhiều thư viện phục vụ xử lý dữ liệu. Trong đề tài, Python được sử dụng chủ yếu để sinh dữ liệu bán hàng mô phỏng với số lượng lớn, hỗ trợ chuẩn bị dữ liệu và xây dựng các chương trình xử lý dữ liệu.

Dữ liệu được tạo bao gồm thông tin khách hàng, sản phẩm, đơn hàng và chi tiết đơn hàng, sau đó được đưa vào PostgreSQL để phục vụ các bước truy vấn, tối ưu và phân tích. Python cũng được sử dụng để xây dựng mã nguồn xử lý dữ liệu với PySpark.

2.2.3. PySpark

PySpark là giao diện Python của Apache Spark, một nền tảng xử lý dữ liệu phân tán. PySpark hỗ trợ các thao tác như đọc dữ liệu, lọc, nhóm, tổng hợp và phân tích dữ liệu trên các tập dữ liệu lớn.

Trong phạm vi đề tài, PySpark được sử dụng để thực hiện các thao tác xử lý và tổng hợp dữ liệu bán hàng, chẳng hạn như phân tích doanh thu theo thời gian, nhóm dữ liệu theo danh mục sản phẩm và xác định các sản phẩm có doanh thu cao.

Việc sử dụng PySpark giúp nhóm tiếp cận phương pháp xử lý dữ liệu lớn và tìm hiểu khả năng mở rộng của các công cụ Big Data bên cạnh việc xử lý dữ liệu trực tiếp bằng SQL. Hiệu quả thực tế phụ thuộc vào cấu hình môi trường và cách triển khai chương trình.

2.2.4. Power BI

Power BI là công cụ phân tích và trực quan hóa dữ liệu, hỗ trợ xây dựng các báo cáo tương tác từ dữ liệu được lưu trữ trong cơ sở dữ liệu hoặc các nguồn dữ liệu khác.

Trong đề tài, Power BI được sử dụng để xây dựng Dashboard phân tích tình hình bán hàng. Các nội dung trực quan hóa dự kiến gồm:

- Doanh thu theo tháng.
- Doanh thu theo danh mục sản phẩm.
- Top 10 sản phẩm có doanh thu cao.
- Các chỉ số tổng hợp liên quan đến đơn hàng và doanh thu.

Dashboard giúp trình bày kết quả phân tích dưới dạng biểu đồ và các chỉ số tổng quan, hỗ trợ người dùng nhận biết xu hướng doanh thu và so sánh hiệu quả kinh doanh giữa các nhóm sản phẩm.

Tóm lại, PostgreSQL, Python, PySpark và Power BI là những công nghệ chính trong hệ thống. PostgreSQL đảm nhiệm lưu trữ và truy vấn; Python hỗ trợ tạo dữ liệu; PySpark phục vụ xử lý và phân tích dữ liệu; Power BI trình bày kết quả trực quan. Sự kết hợp này tạo cơ sở để thực hiện các nội dung tối ưu SQL, tích hợp dữ liệu và phân tích dữ liệu bán hàng quy mô lớn của đề tài.

2.3. Mô hình dữ liệu nguồn và Data Warehouse

2.3.1. Mô hình dữ liệu nguồn

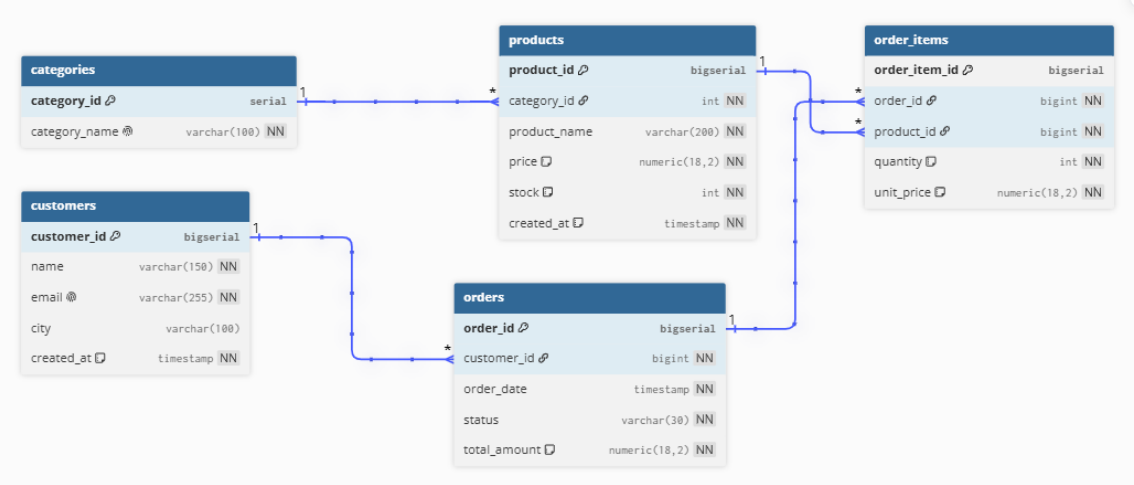
Dữ liệu nguồn của hệ thống được lưu trữ trong PostgreSQL, bao gồm năm bảng chính: categories, products, customers, orders và order_items. Các bảng được thiết kế theo mô hình cơ sở dữ liệu quan hệ, sử dụng khóa chính và khóa ngoại để liên kết dữ liệu, hạn chế sự trùng lặp và đảm bảo tính toàn vẹn.

Cấu trúc các bảng như sau:

Bảng	Nội dung
categories	Lưu thông tin danh mục sản phẩm
products	Lưu thông tin sản phẩm, giá bán và số lượng tồn kho
customers	Lưu thông tin khách hàng và thành phố
orders	Lưu thông tin đơn hàng, ngày đặt hàng và trạng thái
order_items	Lưu chi tiết sản phẩm, số lượng và đơn giá trong từng đơn hàng

Mối quan hệ giữa các bảng được xác định như sau:

- Một danh mục có thể chứa nhiều sản phẩm.
- Một khách hàng có thể phát sinh nhiều đơn hàng.
- Một đơn hàng có thể chứa nhiều chi tiết đơn hàng.
- Một sản phẩm có thể xuất hiện trong nhiều chi tiết đơn hàng.



Hình 2.2. Sơ đồ quan hệ dữ liệu nguồn, minh họa các khóa và quan hệ chính.

Dữ liệu nguồn được sử dụng để thực hiện các truy vấn SQL, đánh giá hiệu năng và làm đầu vào cho quy trình tích hợp dữ liệu. Tổng số bản ghi của năm bảng nguồn khoảng 3,11 triệu bản ghi, tạo điều kiện thực nghiệm các kỹ thuật truy vấn và xử lý dữ liệu.

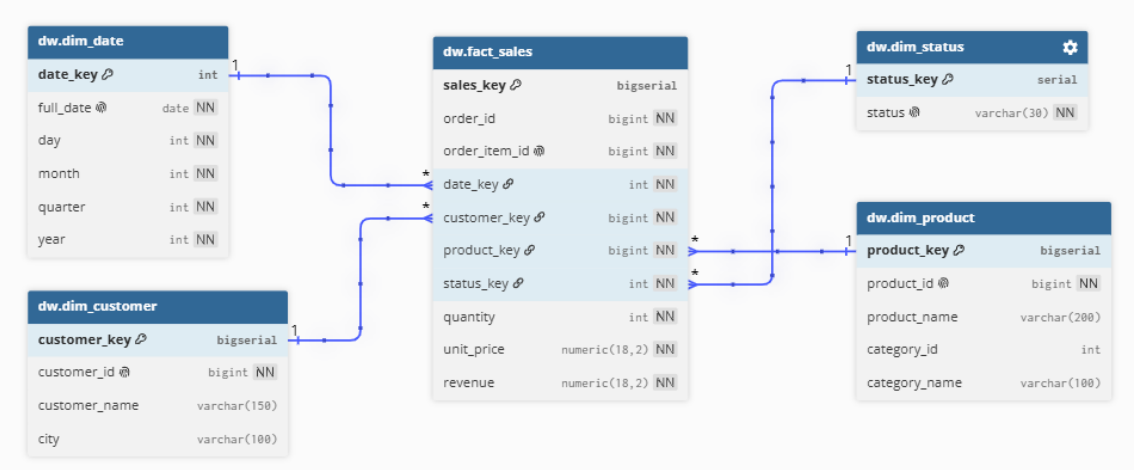
2.3.2. Mô hình Data Warehouse

Data Warehouse (kho dữ liệu) được xây dựng nhằm tổ chức dữ liệu phục vụ mục đích tổng hợp, thống kê và phân tích kinh doanh. Trong đề tài, dữ liệu từ các bảng nguồn được xử lý thông qua quy trình ETL, sau đó lưu vào lược đồ dw trong PostgreSQL.

Kho dữ liệu được tổ chức theo mô hình Star Schema (lược đồ hình sao), gồm một bảng sự kiện trung tâm fact_sales và bốn bảng chiều: dim_date, dim_customer, dim_product, dim_status.

Bảng	Vai trò
fact_sales	Lưu dữ liệu chi tiết bán hàng, số lượng, đơn giá và doanh thu
dim_date	Cung cấp thông tin ngày, tháng, quý và năm
dim_customer	Cung cấp thông tin khách hàng và thành phố
dim_product	Cung cấp thông tin sản phẩm và danh mục
dim_status	Cung cấp trạng thái đơn hàng

Trong đó, fact_sales liên kết với các bảng chiều thông qua những khóa tương ứng như date_key, customer_key, product_key và status_key. Mỗi bản ghi trong bảng sự kiện tương ứng với một chi tiết đơn hàng, giúp phân tích dữ liệu ở mức sản phẩm trong từng đơn hàng.



Hình 2.3. Minh họa mô hình Star Schema của Data Warehouse. Các bảng chiều liên kết với bảng fact thông qua khóa ngoại.

2.3.3. So sánh dữ liệu nguồn và Data Warehouse

Tiêu chí	Dữ liệu nguồn	Data Warehouse
Mục đích	Lưu trữ dữ liệu nghiệp vụ	Phục vụ phân tích, báo cáo
Cấu trúc	Các bảng quan hệ nghiệp vụ	Mô hình Star Schema
Dữ liệu	Thông tin khách hàng, sản phẩm, đơn hàng	Dữ liệu bán hàng đã tích hợp
Khai thác	Truy vấn và quản lý giao dịch	Tổng hợp doanh thu, phân tích theo thời gian, khách hàng và sản phẩm

Việc xây dựng Data Warehouse giúp đơn giản hóa các truy vấn phân tích và tạo nguồn dữ liệu thống nhất cho PySpark và Power BI. Trong quá trình ETL, nhóm kiểm tra số lượng bản ghi và tổng doanh thu để đối chiếu dữ liệu nguồn với dữ liệu sau khi nạp vào kho dữ liệu. Kết quả kiểm tra cho thấy bảng fact_sales có 2.000.011 bản ghi, số lượng bản ghi và tổng doanh thu khớp với dữ liệu nguồn được sử dụng trong quá trình đối chiếu.

3. XÂY DỰNG CƠ SỞ DỮ LIỆU

3.1. Thiết kế các bảng và mối quan hệ

3.1.1. Thiết kế cơ sở dữ liệu

Hệ thống sử dụng PostgreSQL để xây dựng cơ sở dữ liệu bán hàng có tên bigdata_sql. Cơ sở dữ liệu gồm năm bảng chính: categories, products, customers, orders và order_items.

Các bảng được thiết kế theo mô hình quan hệ, sử dụng khóa chính (Primary Key - PK) để xác định duy nhất mỗi bản ghi và khóa ngoại (Foreign Key - FK) để liên kết dữ liệu giữa các bảng.

Bảng	**Các trường chính**	**Chức năng**
categories	category_id, category_name	Lưu danh mục sản phẩm
products	product_id, category_id, product_name, price, stock, created_at	Lưu thông tin sản phẩm
customers	customer_id, name, email, city, created_at	Lưu thông tin khách hàng
orders	order_id, customer_id, order_date, status, total_amount	Lưu thông tin đơn hàng
order_items	order_item_id, order_id, product_id, quantity, unit_price	Lưu chi tiết từng sản phẩm trong đơn hàng

Các mối quan hệ chính:

- categories – products: một danh mục có nhiều sản phẩm.
- customers – orders: một khách hàng có thể có nhiều đơn hàng.
- orders – order_items: một đơn hàng có nhiều dòng chi tiết.
- products – order_items: một sản phẩm có thể xuất hiện trong nhiều dòng chi tiết đơn hàng.

3.1.2. Code tạo cơ sở dữ liệu và các bảng

Đầu tiên, tạo cơ sở dữ liệu trong pgAdmin 4 hoặc chạy câu lệnh sau trong Query Tool đang kết nối tới database postgres:

CREATE DATABASE bigdata_sql;

Sau đó, kết nối tới database bigdata_sql và tạo các bảng bằng SQL dưới đây. Nếu các bảng đã tồn tại và đang chứa dữ liệu, không chạy lại đoạn tạo bảng này vì có thể gây lỗi trùng tên.

```sql
-- 1. Bảng danh mục sản phẩm
CREATE TABLE categories (
    category_id INT PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL
);

-- 2. Bảng sản phẩm
CREATE TABLE products (
    product_id INT PRIMARY KEY,
    category_id INT REFERENCES categories(category_id),
    product_name VARCHAR(200) NOT NULL,
    price NUMERIC(15, 2) NOT NULL,
    stock INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. Bảng khách hàng
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(150) NOT NULL,
    email VARCHAR(200),
    city VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 4. Bảng đơn hàng
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT REFERENCES customers(customer_id),
    order_date TIMESTAMP NOT NULL,
    status VARCHAR(30) NOT NULL,
    total_amount NUMERIC(18, 2)
);

-- 5. Bảng chi tiết đơn hàng
CREATE TABLE order_items (
    order_item_id BIGINT PRIMARY KEY,
    order_id INT REFERENCES orders(order_id),
    product_id INT REFERENCES products(product_id),
    quantity INT NOT NULL,
    unit_price NUMERIC(15, 2) NOT NULL
);
```

3.1.3. Code kiểm tra các bảng đã tạo

Sau khi tạo bảng, sử dụng câu lệnh sau để kiểm tra số lượng bản ghi hiện có trong từng bảng:

```
SELECT 'categories' AS table_name, COUNT(*) AS total_rows
FROM categories
UNION ALL
SELECT 'products', COUNT(*) FROM products
UNION ALL
SELECT 'customers', COUNT(*) FROM customers
UNION ALL
SELECT 'orders', COUNT(*) FROM orders
UNION ALL
SELECT 'order_items', COUNT(*) FROM order_items;
```

Kết quả trên cơ sở dữ liệu của dự án:

	table_name text	total_rows bigint
1	categories	5
2	products	10000
3	customers	100000
4	orders	1000000
5	order_items	2000011

Hình 3.1. Kết quả kiểm tra số lượng bản ghi trong các bảng dữ liệu nguồn.

3.2. Sinh dữ liệu bán hàng quy mô lớn

3.2.1. Phương pháp sinh dữ liệu

PostgreSQL cung cấp hàm generate_series() để tạo một dãy số liên tục. Kết hợp với INSERT INTO ... SELECT, hệ thống có thể sinh hàng nghìn hoặc hàng triệu bản ghi trực tiếp trong cơ sở dữ liệu.

Toàn bộ các câu lệnh bên dưới được chạy trong pgAdmin 4 → Query Tool của database bigdata_sql.

Dữ liệu được tạo theo các mối quan hệ giữa các bảng, bảo đảm các mã khách hàng, sản phẩm và đơn hàng tham chiếu đến các bản ghi tương ứng. Sau khi sinh, dữ liệu được nạp vào PostgreSQL để phục vụ các bước truy vấn, tối ưu và xây dựng Data Warehouse.

Quy mô dữ liệu được sử dụng trong dự án:

- 5 danh mục sản phẩm.
- 10.000 sản phẩm.
- 100.000 khách hàng.
- 1.000.000 đơn hàng.
- 2.000.011 chi tiết đơn hàng.

3.2.2. Sinh dữ liệu danh mục sản phẩm

```
INSERT INTO categories (category_name)
VALUES
    ('Điện thoại'),
    ('Laptop'),
    ('Phụ kiện'),
    ('Đồng hồ'),
    ('Thiết bị gia dụng');
```

3.2.3. Sinh 10.000 sản phẩm

```
INSERT INTO products
    (category_id, product_name, price, stock, created_at)
SELECT
    (gs % 5) + 1,
    'Product_' || gs,
    ROUND((random() * 29900000 + 100000)::NUMERIC, 2),
    FLOOR(random() * 1001)::INT,
    TIMESTAMP '2024-01-01'
        + random() * INTERVAL '730 days'
FROM generate_series(1, 10000) AS gs;
```

Đoạn SQL tạo 10.000 sản phẩm, phân bổ vào 5 danh mục, với giá bán, số lượng tồn kho và ngày tạo được sinh tự động.

3.2.4. Sinh 100.000 khách hàng

```
INSERT INTO customers
    (name, email, city, created_at)
SELECT
    'Customer_' || gs,
    'customer' || gs || '@example.com',
    (ARRAY['Hà Nội', 'Đà Nẵng', 'TP HCM'])
        [1 + FLOOR(random() * 3)::INT],
    TIMESTAMP '2024-01-01'
        + random() * INTERVAL '730 days'
FROM generate_series(1, 100000) AS gs;
```

Mỗi khách hàng có mã định danh tự động, tên, email duy nhất, thành phố và ngày tạo.

3.2.5. Sinh 1.000.000 đơn hàng

```
INSERT INTO orders
    (customer_id, order_date, status, total_amount)
SELECT
    FLOOR(random() * 100000)::INT + 1,
    TIMESTAMP '2024-01-01'
        + random() * INTERVAL '730 days',
    CASE
        WHEN random() < 0.6 THEN 'COMPLETED'
        WHEN random() < 0.5 THEN 'CANCELLED'
        ELSE 'PENDING'
    END,
    ROUND((random() * 50000000 + 100000)::NUMERIC, 2)
FROM generate_series(1, 1000000);
```

Để đảm bảo tỷ lệ trạng thái được phân bổ đúng 60% – 20% – 20%, nên sử dụng cách sinh trạng thái theo số thứ tự thay vì đoạn `CASE` dựa trên nhiều lần gọi `random()` ở trên. Có thể dùng phiên bản sau để thay thế:

```
INSERT INTO orders
    (customer_id, order_date, status, total_amount)
SELECT
    FLOOR(random() * 100000)::INT + 1,
    TIMESTAMP '2024-01-01'
        + random() * INTERVAL '730 days',
    CASE
        WHEN gs % 10 < 6 THEN 'COMPLETED'
        WHEN gs % 10 < 8 THEN 'CANCELLED'
        ELSE 'PENDING'
    END,
    ROUND((random() * 50000000 + 100000)::NUMERIC, 2)
FROM generate_series(1, 1000000) AS gs;
```

Chỉ chạy một trong hai phiên bản sinh đơn hàng, không chạy cả hai vì sẽ tạo thêm dữ liệu trùng về mặt nghiệp vụ. Phiên bản thứ hai phân bổ trạng thái chính xác theo tỷ lệ 60% – 20% – 20%.

3.2.6. Sinh 2.000.011 dòng chi tiết đơn hàng

```
INSERT INTO order_items
    (order_id, product_id, quantity, unit_price)
SELECT
    LEAST(((gs + 1) / 2), 1000000)::INT,
    ((gs * 7919) % 10000 + 1)::INT,
    (gs % 5 + 1)::INT,
    p.price
FROM generate_series(1, 2000011) AS gs
JOIN products p
    ON p.product_id = ((gs * 7919) % 10000 + 1)::INT;
```

Đoạn SQL tạo 2.000.011 dòng chi tiết đơn hàng, liên kết với các đơn hàng và sản phẩm đã tạo. Mỗi dòng có số lượng mua và đơn giá tương ứng với sản phẩm.

3.2.7. Code kiểm tra dữ liệu sau khi sinh

Vì dữ liệu đã được tạo và lưu trong PostgreSQL, có thể demo trực tiếp số lượng bản ghi bằng SQL:

```
-- Kiểm tra dữ liệu danh mục
SELECT * FROM categories LIMIT 5;

-- Kiểm tra dữ liệu sản phẩm
SELECT * FROM products LIMIT 5;

-- Kiểm tra dữ liệu khách hàng
SELECT * FROM customers LIMIT 5;

-- Kiểm tra dữ liệu đơn hàng
SELECT * FROM orders LIMIT 5;

-- Kiểm tra chi tiết đơn hàng
SELECT * FROM order_items LIMIT 5;
```

Để thống kê trạng thái đơn hàng, sử dụng câu lệnh:

```
SELECT
    status,
    COUNT(*) AS total_orders
FROM orders
GROUP BY status
ORDER BY total_orders DESC;
```

Kết quả đã ghi nhận trong dự án:

	status character varying (30)	total_orders bigint
1	COMPLETED	600394
2	CANCELLED	200030
3	PENDING	199576

Hình 3.2. Thống kê số lượng đơn hàng theo trạng thái.

Kết quả cho thấy dữ liệu mô phỏng có nhiều trạng thái đơn hàng, phù hợp để thực hiện các truy vấn thống kê và phân tích doanh thu. Các số liệu trên là kết quả đã ghi nhận của dự án, không phải kết quả tự chạy từ đoạn code minh họa.

3.3. Kiểm tra chất lượng và tính toàn vẹn dữ liệu

3.3.1. Kiểm tra dữ liệu thiếu

Dữ liệu thiếu có thể ảnh hưởng đến kết quả thống kê và phân tích. Vì vậy, nhóm kiểm tra một số trường quan trọng như tên khách hàng, tên sản phẩm, ngày đặt hàng và trạng thái đơn hàng.

```
-- Kiểm tra khách hàng thiếu tên
SELECT COUNT(*) AS missing_customer_name
FROM customers
WHERE name IS NULL;

-- Kiểm tra sản phẩm thiếu tên
SELECT COUNT(*) AS missing_product_name
FROM products
WHERE product_name IS NULL;

-- Kiểm tra đơn hàng thiếu ngày hoặc trạng thái
SELECT COUNT(*) AS invalid_orders
FROM orders
WHERE order_date IS NULL
   OR status IS NULL;
```

Nếu kết quả của các câu truy vấn bằng 0, có nghĩa là không phát hiện bản ghi thiếu dữ liệu tại những trường được kiểm tra. Kết luận này chỉ áp dụng cho các trường trong điều kiện truy vấn.

3.3.2. Kiểm tra tính toàn vẹn khóa ngoại

Khóa ngoại giúp bảo đảm các bản ghi liên kết giữa các bảng có dữ liệu tham chiếu hợp lệ. Có thể kiểm tra các đơn hàng không tìm thấy khách hàng tương ứng như sau:

```
-- Đơn hàng không có khách hàng tương ứng
SELECT COUNT(*) AS orders_without_customer
FROM orders o
LEFT JOIN customers c
    ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;

-- Chi tiết đơn hàng không có sản phẩm tương ứng
SELECT COUNT(*) AS items_without_product
FROM order_items oi
LEFT JOIN products p
    ON oi.product_id = p.product_id
WHERE p.product_id IS NULL;

-- Chi tiết đơn hàng không có đơn hàng tương ứng
SELECT COUNT(*) AS items_without_order
FROM order_items oi
LEFT JOIN orders o
    ON oi.order_id = o.order_id
WHERE o.order_id IS NULL;
```

Kết quả bằng 0 cho thấy không phát hiện bản ghi mồ côi trong từng mối quan hệ được kiểm tra. Đây là bước quan trọng để bảo đảm dữ liệu có thể được kết hợp chính xác bằng các phép JOIN.

3.3.3. Kiểm tra dữ liệu trùng lặp và giá trị không hợp lệ

Nhóm tiếp tục kiểm tra tính duy nhất của email khách hàng và các giá trị số lượng, đơn giá trong chi tiết đơn hàng.

```
-- Kiểm tra email bị trùng
SELECT email, COUNT(*) AS occurrences
FROM customers
WHERE email IS NOT NULL
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY occurrences DESC
LIMIT 10;

-- Kiểm tra số lượng hoặc đơn giá không hợp lệ
SELECT COUNT(*) AS invalid_order_items
FROM order_items
WHERE quantity <= 0
   OR unit_price < 0;

-- Kiểm tra trùng mã chi tiết đơn hàng
SELECT order_item_id, COUNT(*) AS occurrences
FROM order_items
GROUP BY order_item_id
HAVING COUNT(*) > 1
LIMIT 10;
```

Các truy vấn trên giúp phát hiện email trùng, số lượng hoặc đơn giá không hợp lệ và mã chi tiết đơn hàng bị lặp. Nếu truy vấn không trả về bản ghi bất thường hoặc số lượng vi phạm bằng 0, dữ liệu đạt yêu cầu đối với các tiêu chí tương ứng. Email trùng không nhất thiết luôn là lỗi nghiệp vụ, vì vậy cần xem xét quy tắc dữ liệu của hệ thống trước khi loại bỏ.

3.3.4. Tổng hợp kết quả kiểm tra

Trong dự án, nhóm sử dụng các truy vấn SQL để kiểm tra số lượng bản ghi, tính hợp lệ của quan hệ giữa các bảng và tính nhất quán của dữ liệu. Các kiểm tra này giúp phát hiện lỗi trước khi thực hiện tối ưu SQL và ETL.

Mã SQL tính tổng doanh thu:

```
SELECT SUM(quantity * unit_price) AS total_revenue
FROM order_items;
```

	total_revenue numeric
1	139781244448514.84

Hình 3.3. Tổng doanh thu

Kết quả đối chiếu dữ liệu đã ghi nhận cho thấy bảng order_items có 2.000.011 bản ghi. Ở giai đoạn kiểm tra kho dữ liệu, số lượng bản ghi trong dw.fact_sales khớp với nguồn chi tiết đơn hàng và tổng doanh thu đối chiếu là 139781244448514.84. Tuy nhiên, các kết quả về dữ liệu thiếu, email trùng và giá trị không hợp lệ cần được xác nhận bằng cách chạy các câu lệnh kiểm tra tương ứng trước khi kết luận toàn bộ dữ liệu nguồn sạch.

4. TRUY VẤN VÀ TỐI ƯU SQL

Trong đề tài, SQL được sử dụng để khai thác dữ liệu bán hàng, tổng hợp doanh thu và đánh giá hiệu năng truy vấn trên PostgreSQL. Nhóm tập trung vào ba nội dung chính: sử dụng các kỹ thuật SQL nâng cao, phân tích kế hoạch thực thi bằng EXPLAIN ANALYZE và đánh giá hiệu quả tối ưu thông qua chỉ mục INDEX.

Các thử nghiệm được thực hiện trên cơ sở dữ liệu bigdata_sql với khoảng 3,11 triệu bản ghi trong năm bảng dữ liệu nguồn.

4.1. Truy vấn SQL nâng cao: JOIN, CTE, Window Functions

4.1.1. Truy vấn JOIN

JOIN được sử dụng để kết hợp dữ liệu từ nhiều bảng có quan hệ với nhau. Trong hệ thống bán hàng, JOIN giúp kết hợp thông tin đơn hàng, khách hàng, sản phẩm và chi tiết đơn hàng để phục vụ thống kê.

Ví dụ: Thống kê doanh thu theo danh mục sản phẩm đối với đơn hàng đã hoàn thành.

```
SELECT
    c.category_name,
    SUM(oi.quantity * oi.unit_price) AS revenue,
    SUM(oi.quantity) AS total_quantity
FROM order_items oi
JOIN products p
    ON oi.product_id = p.product_id
JOIN categories c
    ON p.category_id = c.category_id
JOIN orders o
    ON oi.order_id = o.order_id
WHERE o.status = 'COMPLETED'
GROUP BY c.category_name
ORDER BY revenue DESC;
```

Giải thích:

- JOIN: kết hợp bốn bảng theo các khóa liên kết.
- WHERE: chỉ lấy các đơn hàng có trạng thái COMPLETED.
- SUM: tính doanh thu và tổng số lượng sản phẩm.
- GROUP BY: nhóm dữ liệu theo danh mục.
- ORDER BY: sắp xếp danh mục theo doanh thu giảm dần.

Kết quả giúp xác định danh mục sản phẩm đóng góp doanh thu cao nhất.

4.1.2. Truy vấn CTE

CTE (Common Table Expression) cho phép tạo một tập kết quả tạm thời bằng mệnh đề WITH, giúp truy vấn phức tạp dễ đọc và dễ bảo trì hơn.

Ví dụ: Tính doanh thu theo tháng và xếp hạng các tháng có doanh thu cao.

```
WITH monthly_revenue AS (
    SELECT
        DATE_TRUNC('month', o.order_date) AS sales_month,
        SUM(oi.quantity * oi.unit_price) AS revenue
    FROM orders o
    JOIN order_items oi
        ON o.order_id = oi.order_id
    WHERE o.status = 'COMPLETED'
    GROUP BY DATE_TRUNC('month', o.order_date)
)
SELECT
    sales_month,
    revenue
FROM monthly_revenue
ORDER BY revenue DESC;
```

Giải thích:

- CTE monthly_revenue tính doanh thu theo tháng.
- DATE_TRUNC('month', ...) đưa ngày đặt hàng về đầu tháng tương ứng.
- Truy vấn bên ngoài sắp xếp các tháng theo doanh thu giảm dần.

CTE giúp tách phần tổng hợp dữ liệu và phần truy vấn kết quả thành các bước rõ ràng.

4.1.3. Window Functions

Window Functions cho phép tính toán trên một nhóm dòng liên quan mà vẫn giữ lại từng dòng kết quả. Trong đề tài, kỹ thuật này có thể được sử dụng để xếp hạng sản phẩm theo doanh thu.

Ví dụ: Xếp hạng 10 sản phẩm có doanh thu cao nhất trong các đơn hàng đã hoàn thành.

```
WITH product_revenue AS (
    SELECT
        p.product_id,
        p.product_name,
        SUM(oi.quantity * oi.unit_price) AS revenue
    FROM products p
    JOIN order_items oi
        ON p.product_id = oi.product_id
    JOIN orders o
        ON oi.order_id = o.order_id
    WHERE o.status = 'COMPLETED'
```

```
    GROUP BY p.product_id, p.product_name
),
ranked_products AS (
    SELECT
        product_id,
        product_name,
        revenue,
        RANK() OVER (
            ORDER BY revenue DESC
        ) AS revenue_rank
    FROM product_revenue
)
SELECT *
FROM ranked_products
WHERE revenue_rank <= 10
ORDER BY revenue_rank;
```

Giải thích:

- SUM: tính doanh thu của từng sản phẩm.
- RANK() OVER (...): xếp hạng sản phẩm theo doanh thu.
- WHERE revenue_rank <= 10: lấy các sản phẩm nằm trong top 10 thứ hạng.

Do RANK() có thể tạo đồng hạng, truy vấn có thể trả về nhiều hơn 10 dòng nếu có sản phẩm bằng thứ hạng cuối cùng. Nếu cần đúng 10 dòng, có thể thay RANK() bằng ROW_NUMBER().

Các truy vấn trên giúp khai thác dữ liệu bán hàng ở nhiều cấp độ: danh mục, thời gian và sản phẩm, đồng thời minh họa khả năng kết hợp dữ liệu và tổng hợp bằng SQL nâng cao.

4.2. Phân tích kế hoạch thực thi bằng EXPLAIN ANALYZE

4.2.1. Khái niệm

EXPLAIN ANALYZE là câu lệnh trong PostgreSQL dùng để thực thi truy vấn và hiển thị kế hoạch thực thi cùng thời gian thực tế. Công cụ này giúp xác định các thao tác tốn thời gian, chẳng hạn như quét toàn bộ bảng, JOIN nhiều bảng hoặc sắp xếp dữ liệu lớn.

Một số thành phần quan trọng trong kết quả:

- Seq Scan: quét tuần tự các bản ghi của bảng.
- Index Scan: truy xuất dữ liệu thông qua chỉ mục.
- Hash Join / Nested Loop: các phương pháp kết hợp dữ liệu giữa các bảng.
- Sort: sắp xếp dữ liệu.
- Execution Time: tổng thời gian thực thi truy vấn, tính bằng mili giây (ms).

4.2.2. Code phân tích kế hoạch thực thi

Ví dụ, phân tích truy vấn tính doanh thu theo tháng từ dữ liệu đơn hàng và chi tiết đơn hàng:

```
EXPLAIN ANALYZE
SELECT
    DATE_TRUNC('month', o.order_date) AS sales_month,
    SUM(oi.quantity * oi.unit_price) AS revenue
FROM orders o
JOIN order_items oi
    ON o.order_id = oi.order_id
WHERE o.status = 'COMPLETED'
GROUP BY DATE_TRUNC('month', o.order_date)
ORDER BY sales_month;
```

Cách thực hiện demo:

1. Mở pgAdmin 4 và kết nối tới database bigdata_sql.
2. Mở Query Tool, dán câu lệnh trên và nhấn Execute.
3. Quan sát phần kế hoạch thực thi được trả về.
4. Ghi lại Execution Time, các thao tác quét bảng và sắp xếp dữ liệu.

Trong lần phân tích trước của dự án, truy vấn tổng hợp theo tháng có sử dụng Seq Scan trên bảng orders và thực hiện sắp xếp ngoài bộ nhớ (external merge) với dữ liệu trung gian ghi ra đĩa. Điều này cho thấy chi phí xử lý không chỉ phụ thuộc vào việc tìm bản ghi mà còn phụ thuộc vào khối lượng dữ liệu cần tổng hợp và sắp xếp.

	QUERY PLAN text
6	Sort Method: external merge Disk: 36456kB
7	Buffers: shared hit=755 read=25261, temp read=20825 written=20844
8	-> Hash Join (cost=32190.34..103426.85 rows=1190141 width=21) (actual time=318.892..2303.429 rows=1200797.00 loops=1)
9	Hash Cond: (oi.order_id = o.order_id)
10	Buffers: shared hit=752 read=25261, temp read=11714 written=11714
11	-> Seq Scan on order_items oi (cost=0.00..36667.11 rows=2000011 width=21) (actual time=0.156..368.265 rows=2000011.00 loo…
12	Buffers: shared hit=376 read=16291
13	-> Hash (cost=21846.00..21846.00 rows=595067 width=16) (actual time=314.120..314.121 rows=600394.00 loops=1)
14	Buckets: 262144 Batches: 8 Memory Usage: 5570kB
15	Buffers: shared hit=376 read=8970, temp written=2305
16	-> Seq Scan on orders o (cost=0.00..21846.00 rows=595067 width=16) (actual time=0.416..171.473 rows=600394.00 loops=1)
17	Filter: ((status)::text = 'COMPLETED'::text)
18	Rows Removed by Filter: 399606
19	Buffers: shared hit=376 read=8970
20	Planning:
21	Buffers: shared hit=16
22	Planning Time: 0.714 ms
23	Execution Time: 3312.580 ms

Hình 4.1. Kết quả EXPLAIN ANALYZE của truy vấn doanh thu theo tháng.

4.3. Tối ưu bằng INDEX và đánh giá hiệu năng

4.3.1. Mục tiêu tối ưu

INDEX (chỉ mục) giúp PostgreSQL tìm kiếm dữ liệu nhanh hơn mà không cần quét toàn bộ bảng trong một số truy vấn. Trong dự án, các chỉ mục được tạo trên những cột thường sử dụng để JOIN, lọc dữ liệu và truy vấn theo thời gian.

4.3.2. Tạo INDEX

Thực hiện các câu lệnh sau trong pgAdmin 4 → Query Tool:

```
CREATE INDEX idx_orders_customer_id
ON orders(customer_id);

CREATE INDEX idx_orders_status
ON orders(status);

CREATE INDEX idx_orders_order_date
ON orders(order_date);

CREATE INDEX idx_order_items_order_id
ON order_items(order_id);

CREATE INDEX idx_order_items_product_id
ON order_items(product_id);
```

Các chỉ mục trên hỗ trợ truy vấn khách hàng, trạng thái đơn hàng, thời gian đặt hàng và liên kết giữa đơn hàng với sản phẩm.

4.3.3. Đo lường hiệu năng truy vấn

Sử dụng EXPLAIN ANALYZE để xem kế hoạch thực thi và thời gian chạy thực tế của PostgreSQL.

Ví dụ truy vấn doanh thu theo trạng thái đơn hàng:

```
EXPLAIN ANALYZE
SELECT o.status,
       SUM(oi.quantity * oi.unit_price) AS revenue
FROM orders o
JOIN order_items oi
    ON o.order_id = oi.order_id
GROUP BY o.status;
```

Thực hiện truy vấn trước và sau khi tạo INDEX, ghi lại thời gian Execution Time để so sánh.

4.3.4. Kết quả đánh giá hiệu năng

Truy vấn	Trước INDEX (ms)	Sau INDEX (ms)	Cải thiện
Q1	0,099	0,084	15,15%
Q2	146,994	117,393	20,14%
Q3	17,504	16,085	8,68%
Q4	0,296	0,289	2,36%

Công thức tính tỷ lệ cải thiện:

$$\text{Cải thiện (\%)} = \frac{T_{\text{trước}} - T_{\text{sau}}}{T_{\text{trước}}} \times 100$$

Nhận xét: Cả bốn truy vấn đều có thời gian thực thi giảm sau khi tạo INDEX. Q2 cải thiện nhiều nhất, khoảng 20,14%, trong khi Q4 cải thiện ít nhất, khoảng 2,36%. Kết quả cho thấy INDEX có thể giúp tăng tốc truy vấn, nhưng mức độ hiệu quả phụ thuộc vào điều kiện truy vấn và dữ liệu.

Kết luận chương 4: Nhóm đã sử dụng JOIN, CTE và Window Functions để khai thác dữ liệu bán hàng; sử dụng EXPLAIN ANALYZE để phân tích kế hoạch thực thi; đồng thời thử nghiệm các chỉ mục trên những cột quan trọng. Kết quả cho thấy việc tối ưu SQL cần kết hợp giữa thiết kế truy vấn, phân tích kế hoạch thực thi và đo lường hiệu năng thực tế.

5. TÍCH HỢP DỮ LIỆU VÀ DATA WAREHOUSE

Trong đề tài, Data Warehouse được xây dựng nhằm tập hợp và tổ chức dữ liệu bán hàng từ cơ sở dữ liệu nguồn để phục vụ truy vấn phân tích và trực quan hóa. Nhóm sử dụng PostgreSQL để lưu trữ dữ liệu nguồn và kho dữ liệu, đồng thời xây dựng quy trình ETL để trích xuất, chuyển đổi và nạp dữ liệu.

Quá trình tích hợp dữ liệu gồm ba nội dung chính: thiết kế mô hình Star Schema, xây dựng quy trình ETL thông qua khu vực Staging và kiểm tra tính đầy đủ, nhất quán của dữ liệu sau khi nạp vào kho dữ liệu.

5.1. Thiết kế Star Schema: Fact và Dimension

5.1.1. Khái niệm Star Schema

Star Schema (mô hình hình sao) là mô hình tổ chức dữ liệu thường được sử dụng trong Data Warehouse. Mô hình gồm một bảng Fact ở trung tâm, lưu các sự kiện và số liệu cần phân tích, cùng các bảng Dimension lưu thông tin mô tả để phân loại và tổng hợp dữ liệu.

Trong đề tài, nhóm sử dụng mô hình Star Schema để phân tích doanh thu theo thời gian, khách hàng, sản phẩm và trạng thái đơn hàng.

5.1.2. Thiết kế các bảng Fact và Dimension

Kho dữ liệu được xây dựng trong schema dw của PostgreSQL, gồm năm bảng chính:

Bảng	**Loại**	**Chức năng**
fact_sales	Fact	Lưu chi tiết bán hàng, số lượng, đơn giá và doanh thu
dim_date	Dimension	Lưu ngày, tháng, quý và năm
dim_customer	Dimension	Lưu thông tin khách hàng và thành phố
dim_product	Dimension	Lưu thông tin sản phẩm và danh mục
dim_status	Dimension	Lưu trạng thái đơn hàng

Bảng Fact - dw.fact_sales. Đây là bảng trung tâm của kho dữ liệu. Mỗi bản ghi tương ứng với một dòng chi tiết đơn hàng, chứa các khóa liên kết đến bảng chiều và các chỉ số phục vụ phân tích.

Các trường quan trọng gồm:

- sales_key: khóa chính của bảng Fact.
- order_id, order_item_id: mã đơn hàng và mã chi tiết đơn hàng nguồn.
- date_key, customer_key, product_key, status_key: các khóa liên kết đến bảng Dimension.
- quantity: số lượng sản phẩm.
- unit_price: đơn giá sản phẩm.
- revenue: doanh thu của dòng chi tiết đơn hàng.

Trong phạm vi đề tài, doanh thu của một dòng chi tiết được xác định bằng số lượng nhân với đơn giá.

Các bảng Dimension

- dim_date: hỗ trợ tổng hợp doanh thu theo ngày, tháng, quý và năm.
- dim_customer: hỗ trợ phân tích theo khách hàng và thành phố.
- dim_product: hỗ trợ phân tích theo sản phẩm và danh mục sản phẩm.
- dim_status: hỗ trợ phân tích dữ liệu theo trạng thái đơn hàng.

Việc tách dữ liệu thành bảng Fact và Dimension giúp tổ chức dữ liệu phù hợp với các truy vấn phân tích và báo cáo.

5.1.3. Code truy vấn kiểm tra Data Warehouse

Để kiểm tra cấu trúc và số lượng dữ liệu trong kho, nhóm sử dụng các câu lệnh SQL sau trong pgAdmin 4.

```
-- Kiểm tra số bản ghi trong các bảng Data Warehouse
SELECT 'dim_date' AS table_name, COUNT(*) AS total_rows
FROM dw.dim_date
UNION ALL
SELECT 'dim_customer', COUNT(*) FROM dw.dim_customer
UNION ALL
SELECT 'dim_product', COUNT(*) FROM dw.dim_product
UNION ALL
SELECT 'dim_status', COUNT(*) FROM dw.dim_status
UNION ALL
SELECT 'fact_sales', COUNT(*) FROM dw.fact_sales;
```

	table_name text	total_rows bigint
1	dim_date	974
2	dim_custom...	100000
3	dim_product	10000
4	dim_status	3
5	fact_sales	2000011

Hình 5.1. kiểm tra Data Warehouse

Để minh họa mối liên kết giữa bảng Fact và Dimension, thực hiện truy vấn doanh thu theo danh mục sản phẩm:

```
SELECT
    dp.category_name,
    SUM(fs.revenue) AS total_revenue
FROM dw.fact_sales fs
JOIN dw.dim_product dp
    ON fs.product_key = dp.product_key
JOIN dw.dim_status ds
    ON fs.status_key = ds.status_key
WHERE ds.status = 'COMPLETED'
GROUP BY dp.category_name
ORDER BY total_revenue DESC;
```

	category_name character varying (100)	total_revenue numeric
1	Laptop	16915743330394.46
2	Đồng hồ	16864234029394.66
3	Phụ kiện	16805382173138.78
4	Điện thoại	16707896815995.72
5	Tai nghe	16593535987666.30

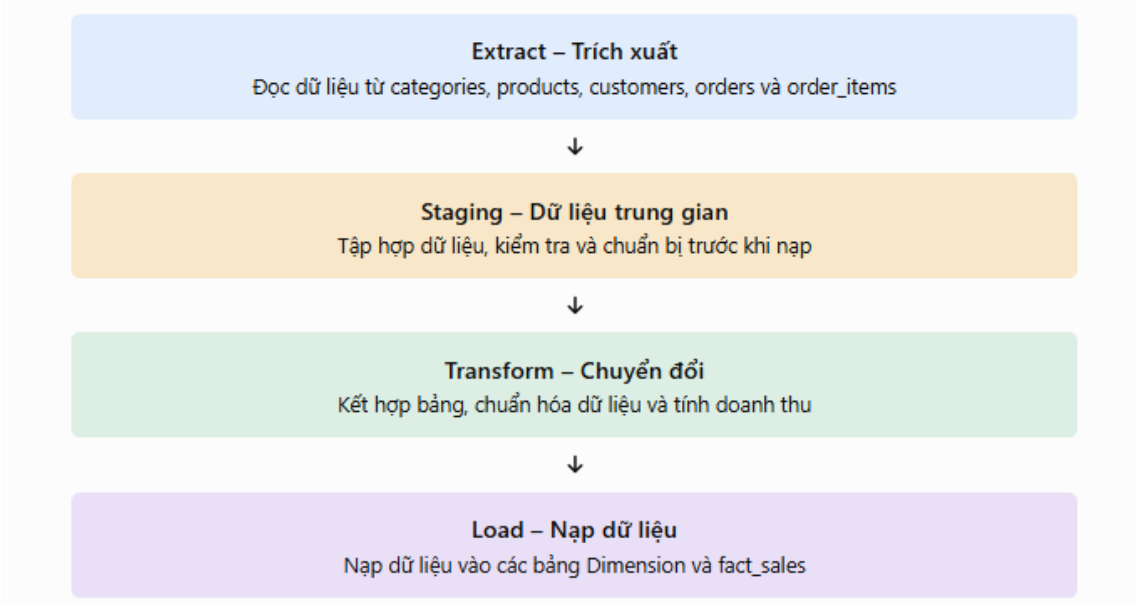
Hình 5.2. Truy vấn doanh thu sản phẩm

Truy vấn kết hợp bảng Fact với các bảng Dimension để xác định doanh thu theo từng danh mục, chỉ tính các đơn hàng đã hoàn thành. Kết quả được sử dụng để kiểm tra dữ liệu kho và làm nguồn cho Dashboard.

5.2. Xây dựng quy trình ETL và Staging

5.2.1. Quy trình ETL

ETL là quy trình gồm ba bước: Extract (trích xuất), Transform (chuyển đổi) và Load (nạp dữ liệu). Trong đề tài, ETL được sử dụng để chuyển dữ liệu từ các bảng nguồn sang Data Warehouse.



Hình 5.3. Quy trình ETL của hệ thống.

Bước 1 – Extract: Dữ liệu được đọc từ năm bảng nguồn trong schema public. Các bảng được liên kết thông qua khóa khách hàng, sản phẩm và đơn hàng.

Bước 2 – Staging: Dữ liệu được tập hợp vào khu vực trung gian để thuận tiện kiểm tra, xử lý và đối chiếu trước khi nạp vào kho dữ liệu.

Bước 3 – Transform: Các bảng nguồn được kết hợp để xác định thông tin khách hàng, sản phẩm, ngày đặt hàng và trạng thái. Doanh thu của từng dòng chi tiết được tính bằng quantity × unit_price.

Bước 4 – Load: Dữ liệu sau chuyển đổi được nạp vào các bảng Dimension và bảng Fact trong schema dw. Các khóa liên kết được sử dụng để kết nối dữ liệu sự kiện với thông tin mô tả.

5.2.2. Code demo trích xuất dữ liệu vào Staging

Đoạn SQL sau minh họa cách tạo một bảng Staging tạm thời để tập hợp dữ liệu bán hàng trong tháng 1 và tháng 2 năm 2024. Đây là ví dụ có thể chạy trực tiếp trong Query Tool; bảng tạm sẽ tồn tại trong phiên kết nối hiện tại.

```
-- Tạo bảng staging tạm thời cho dữ liệu tháng 1 và 2/2024
CREATE TEMP TABLE staging_sales_demo AS
SELECT
    o.order_id,
    oi.order_item_id,
    o.order_date,
    o.customer_id,
    oi.product_id,
    o.status,
```

```
    oi.quantity,
    oi.unit_price,
    oi.quantity * oi.unit_price AS revenue
FROM public.orders o
JOIN public.order_items oi
    ON o.order_id = oi.order_id
WHERE o.order_date >= TIMESTAMP '2024-01-01'
  AND o.order_date <  TIMESTAMP '2024-03-01';
```

-- Kiểm tra số lượng bản ghi trong staging

```
SELECT COUNT(*) AS staging_rows
FROM staging_sales_demo;
```

-- Xem một số bản ghi

```
SELECT *
FROM staging_sales_demo
ORDER BY order_date
LIMIT 10;
```

Bảng Staging trên giúp tập hợp thông tin từ đơn hàng và chi tiết đơn hàng thành một tập dữ liệu trung gian. Điều kiện lọc theo khoảng ngày sử dụng mốc đầu tháng 1 và mốc đầu tháng 3, nhờ đó bao gồm toàn bộ ngày trong tháng 1 và tháng 2.

Lưu ý: Đây là đoạn code minh họa bước Extract và chuẩn bị dữ liệu Staging, không thay thế toàn bộ quy trình ETL của dự án. Bảng staging_sales_demo chỉ là bảng tạm để demo, không phải khẳng định tên bảng Staging chính thức đang được sử dụng trong dự án.

5.2.3. Nạp và sử dụng dữ liệu trong Data Warehouse

Sau khi dữ liệu được kiểm tra và chuyển đổi, hệ thống lưu dữ liệu vào các bảng Dimension và fact_sales. Các bảng Dimension cung cấp thông tin mô tả, trong khi bảng Fact lưu số liệu bán hàng phục vụ tổng hợp.

Có thể demo việc truy vấn dữ liệu sau ETL bằng câu lệnh:

-- Thống kê doanh thu theo tháng trong Data Warehouse

```
SELECT
    dd.year,
    dd.month,
    SUM(fs.revenue) AS monthly_revenue,
    COUNT(DISTINCT fs.order_id) AS total_orders
FROM dw.fact_sales fs
JOIN dw.dim_date dd
    ON fs.date_key = dd.date_key
JOIN dw.dim_status ds
    ON fs.status_key = ds.status_key
WHERE ds.status = 'COMPLETED'
GROUP BY dd.year, dd.month
ORDER BY dd.year, dd.month;
```

Truy vấn trên sử dụng dữ liệu đã tích hợp trong Data Warehouse để tổng hợp doanh thu và số đơn hàng theo tháng, phục vụ phân tích xu hướng kinh doanh.

5.3. Kiểm tra, đối chiếu dữ liệu sau ETL

5.3.1. Kiểm tra số lượng bản ghi

Sau khi hoàn thành ETL, nhóm đối chiếu số lượng bản ghi giữa dữ liệu nguồn và bảng Fact. Mục đích là xác định dữ liệu có bị thiếu hoặc phát sinh thêm trong quá trình tích hợp hay không.

-- So sánh số dòng chi tiết đơn hàng nguồn và bảng Fact

```
SELECT
    (SELECT COUNT(*)
     FROM public.order_items) AS source_rows,
    (SELECT COUNT(*)
     FROM dw.fact_sales) AS warehouse_rows,
    (SELECT COUNT(*)
     FROM public.order_items)
    -
    (SELECT COUNT(*)
     FROM dw.fact_sales) AS difference;
```

Kết quả đã ghi nhận trong dự án:

	source_rows bigint	warehouse_rows bigint	difference bigint
1	2000011	2000011	0

Hình 5.4. Số lượng bản ghi

Kết quả cho thấy số lượng bản ghi nguồn và bảng Fact khớp nhau. Đây là một chỉ tiêu quan trọng để đánh giá tính đầy đủ của dữ liệu sau ETL.

5.3.2. Đối chiếu tổng doanh thu

Bên cạnh số lượng bản ghi, nhóm kiểm tra tổng doanh thu giữa dữ liệu nguồn và kho dữ liệu. Doanh thu nguồn được tính bằng tổng số lượng nhân với đơn giá của từng dòng chi tiết đơn hàng.

-- Tổng doanh thu từ dữ liệu nguồn

```
SELECT
    SUM(quantity * unit_price) AS source_revenue
FROM public.order_items;
```

-- Tổng doanh thu trong Data Warehouse

```
SELECT
    SUM(revenue) AS warehouse_revenue
FROM dw.fact_sales;
```

-- Đối chiếu hai nguồn

```
SELECT
    (SELECT SUM(quantity * unit_price)
     FROM public.order_items) AS source_revenue,
    (SELECT SUM(revenue)
     FROM dw.fact_sales) AS warehouse_revenue,
    (SELECT SUM(quantity * unit_price)
     FROM public.order_items)
    -
    (SELECT SUM(revenue)
     FROM dw.fact_sales) AS revenue_difference;
```

Kết quả đối chiếu đã ghi nhận:

	source_revenue numeric	warehouse_revenue numeric	revenue_difference numeric
1	1397812444448514.84	1397812444448514.84	0.00

Hình 5.5. Đối chiếu doanh thu

Kết quả cho thấy tổng doanh thu giữa dữ liệu nguồn và Data Warehouse khớp nhau. Điều này xác nhận tính nhất quán của chỉ tiêu doanh thu trong lần đối chiếu đã thực hiện.

5.3.3. Kiểm tra trùng lặp và dữ liệu không hợp lệ

Để hạn chế tình trạng một dòng chi tiết đơn hàng bị nạp nhiều lần, nhóm kiểm tra tính duy nhất của order_item_id trong bảng Fact.

-- Kiểm tra trùng mã chi tiết đơn hàng trong Data Warehouse

```
SELECT
    order_item_id,
    COUNT(*) AS occurrences
FROM dw.fact_sales
GROUP BY order_item_id
HAVING COUNT(*) > 1
LIMIT 10;
```

-- Kiểm tra doanh thu không khớp số lượng và đơn giá

```
SELECT COUNT(*) AS revenue_mismatches
FROM dw.fact_sales
WHERE revenue IS DISTINCT FROM quantity * unit_price;
```

Nếu truy vấn thứ nhất không trả về dòng nào, không phát hiện mã chi tiết đơn hàng bị trùng. Nếu revenue_mismatches bằng 0, doanh thu trong bảng Fact khớp với phép tính số lượng nhân đơn giá.

5.3.4. Đối chiếu dữ liệu Staging

Trong quá trình kiểm tra dữ liệu trung gian, nhóm đã thực hiện đối chiếu dữ liệu nguồn với Staging cho giai đoạn tháng 1 và tháng 2 năm 2024.

```
SELECT
    (SELECT COUNT(*)
     FROM public.orders
     WHERE order_date >= '2024-01-01'
       AND order_date < '2024-03-01') AS source_count,

    (SELECT COUNT(*)
     FROM dw.staging_orders
     WHERE order_date >= '2024-01-01'
       AND order_date < '2024-03-01') AS staging_count,

    (SELECT COUNT(*)
     FROM public.orders
     WHERE order_date >= '2024-01-01'
       AND order_date < '2024-03-01')
    -
    (SELECT COUNT(*)
     FROM dw.staging_orders
     WHERE order_date >= '2024-01-01'
       AND order_date < '2024-03-01') AS difference;
```

	source_count bigint	staging_count bigint	difference bigint
1	61970	61970	0

Hình 5.6. Đối chiếu dữ liệu Staging

Kết quả cho thấy số lượng bản ghi nguồn và Staging khớp nhau trong giai đoạn đã kiểm tra. Kết quả này giúp xác nhận bước trích xuất dữ liệu của giai đoạn trên, nhưng không thay thế việc kiểm tra toàn bộ các giai đoạn còn lại.

5.3.5. Đánh giá kết quả tích hợp dữ liệu

Qua quá trình xây dựng Data Warehouse và kiểm tra sau ETL, nhóm ghi nhận các kết quả chính:

- Xây dựng kho dữ liệu theo mô hình Star Schema với một bảng Fact và bốn bảng Dimension.
- Tích hợp 2.000.011 dòng chi tiết đơn hàng vào dw.fact_sales.
- Đối chiếu số lượng bản ghi và tổng doanh thu giữa nguồn và kho dữ liệu, ghi nhận chênh lệch bằng 0.
- Kiểm tra dữ liệu Staging trong giai đoạn tháng 1 và tháng 2 năm 2024, với số lượng bản ghi nguồn và Staging đều bằng 61.970.
- Tạo nguồn dữ liệu phục vụ các truy vấn phân tích doanh thu, xử lý bằng PySpark và trực quan hóa trên Power BI.

6. PHÂN TÍCH DỮ LIỆU VÀ DASHBOARD

6.1. Phân tích dữ liệu bằng Apache Spark

6.1.1. Giới thiệu

Sau khi hoàn thành quá trình tích hợp dữ liệu vào Data Warehouse, nhóm tiến hành phân tích dữ liệu bán hàng nhằm khai thác thông tin về doanh thu, số lượng sản phẩm bán ra và tình hình kinh doanh theo thời gian.

Trong dự án, Apache Spark được sử dụng thông qua PySpark để xử lý và tổng hợp dữ liệu từ PostgreSQL. Spark chạy ở chế độ local trên máy tính cá nhân, không triển khai trên cụm máy chủ Spark. Các kết quả phân tích được xuất thành tệp CSV để phục vụ việc xây dựng Dashboard bằng Power BI.

Dữ liệu phân tích được lấy từ bảng dw.fact_sales, kết hợp với các bảng chiều dw.dim_date và dw.dim_product. Quá trình xử lý tập trung vào ba nội dung chính: doanh thu theo tháng, doanh thu theo danh mục sản phẩm và Top 10 sản phẩm có doanh thu cao nhất.

6.1.2. Quy trình phân tích dữ liệu

Quy trình phân tích được thực hiện theo các bước:

1. Đọc dữ liệu bán hàng từ Data Warehouse trong PostgreSQL.
2. Kết hợp bảng dữ liệu sự kiện bán hàng với các bảng chiều để bổ sung thông tin thời gian và sản phẩm.
3. Nhóm và tổng hợp dữ liệu theo tháng, danh mục và sản phẩm.
4. Đối chiếu kết quả tổng hợp với truy vấn SQL trên PostgreSQL.
5. Xuất kết quả thành các tệp CSV để sử dụng trong Power BI.

Các kết quả được lưu trong thư mục Big_Data_Extension/output, gồm monthly_revenue.csv, category_revenue.csv, top_products.csv và validation_summary.txt.

6.1.3. Kết quả phân tích

Theo dữ liệu tổng hợp hiện có, hệ thống ghi nhận 32 tháng dữ liệu, từ tháng 1/2024 đến tháng 8/2026. Tổng doanh thu là 139.781.244.448.514,84 đơn vị tiền tệ theo dữ liệu lưu trữ; tổng số đơn hàng là 1.000.000 và tổng số lượng sản phẩm bán ra là 6.001.039.

Bảng tổng hợp doanh thu theo năm:

	Năm integer	Số tháng bigint	Doanh thu numeric	Số đơn hàng bigint
1	2024	12	52522197038510.42	375821
2	2025	12	52281045682288.28	373931
3	2026	8	34978001727716.14	250248

Hình 6.1. Doanh thu theo năm

Năm 2026 mới có dữ liệu đến tháng 8 nên không thể so sánh trực tiếp tổng doanh thu cả năm với hai năm đã đủ 12 tháng. Trong dữ liệu hiện có, tháng 1/2026 có doanh thu cao nhất, đạt 4.507.514.671.390,18; tháng 2/2025 có doanh thu thấp nhất, đạt 4.022.450.275.862,20.

Để minh họa cách tổng hợp doanh thu theo năm và kiểm tra trực tiếp trên PostgreSQL, nhóm sử dụng truy vấn SQL sau:

```
SELECT
    d.year AS "Năm",
    COUNT(DISTINCT d.month) AS "Số tháng",
    SUM(f.revenue) AS "Doanh thu",
    COUNT(DISTINCT f.order_id) AS "Số đơn hàng"
FROM dw.fact_sales f
JOIN dw.dim_date d
    ON f.date_key = d.date_key
GROUP BY d.year
ORDER BY d.year;
```

Truy vấn thực hiện phép kết hợp bảng dữ liệu bán hàng với bảng chiều thời gian, sau đó tính tổng doanh thu, số đơn hàng và số lượng sản phẩm bán ra theo từng năm.

6.2. Xây dựng Dashboard bằng Power BI

6.2.1. Nguồn dữ liệu

Sau khi hoàn thành quá trình tổng hợp, nhóm sử dụng ba tệp CSV do Spark tạo ra làm nguồn dữ liệu cho Power BI:

- monthly_revenue.csv: doanh thu, số đơn hàng và số lượng bán theo tháng.
- category_revenue.csv: doanh thu và số lượng bán theo danh mục sản phẩm.
- top_products.csv: danh sách 10 sản phẩm có doanh thu cao nhất.

Việc sử dụng dữ liệu đã tổng hợp giúp giảm lượng dữ liệu cần xử lý trực tiếp trong công cụ trực quan hóa, đồng thời thuận tiện cho quá trình cập nhật và kiểm tra kết quả.

6.2.2. Các chỉ tiêu trên Dashboard

Dashboard tập trung thể hiện các chỉ tiêu chính:

- Tổng doanh thu.
- Tổng số đơn hàng.
- Tổng số lượng sản phẩm bán ra.
- Biến động doanh thu theo tháng.
- So sánh doanh thu giữa các danh mục sản phẩm.
- Xếp hạng Top 10 sản phẩm theo doanh thu.

Các chỉ tiêu tổng quan giúp người xem nhanh chóng nắm bắt tình hình bán hàng. Biểu đồ doanh thu theo tháng hỗ trợ theo dõi biến động theo thời gian; biểu đồ danh mục giúp so sánh mức đóng góp của từng nhóm sản phẩm; bảng Top 10 hỗ trợ xác định các sản phẩm có doanh thu cao nhất.

Do tệp top_products.csv chỉ chứa 10 sản phẩm đứng đầu, dữ liệu trong tệp này không đại diện cho toàn bộ danh mục sản phẩm của hệ thống.

6.2.3. Đánh giá Dashboard

Dashboard giúp chuyển đổi dữ liệu bán hàng đã tổng hợp thành các biểu đồ trực quan, hỗ trợ việc theo dõi và so sánh các chỉ tiêu kinh doanh. Việc tách quá trình xử lý dữ liệu bằng Spark khỏi quá trình trực quan hóa bằng Power BI cũng giúp quy trình rõ ràng hơn: Spark thực hiện tổng hợp, PostgreSQL hỗ trợ đối chiếu, còn Power BI đảm nhiệm việc trình bày kết quả.

Khi cập nhật dữ liệu, nhóm cần chạy lại quy trình xử lý để tạo các tệp CSV mới, sau đó làm mới dữ liệu trong Power BI.

6.3. Phân tích kết quả kinh doanh

6.3.1. Phân tích doanh thu theo thời gian

Kết quả phân tích cho thấy doanh thu theo tháng dao động trong khoảng 4,02 đến 4,51 nghìn tỷ đơn vị tiền tệ theo dữ liệu lưu trữ. Tháng 1/2026 có doanh thu cao nhất trong giai đoạn được thống kê, trong khi tháng 2/2025 có doanh thu thấp nhất.

Các kết quả này cung cấp cơ sở để quan sát biến động doanh thu giữa các tháng. Tuy nhiên, do dữ liệu năm 2026 chưa đầy đủ và chưa phân tích các yếu tố như mùa vụ, chi phí hoặc chương trình khuyến mại, chưa thể kết luận nguyên nhân làm doanh thu tăng hoặc giảm.

6.3.2. Phân tích doanh thu theo danh mục

Dữ liệu hiện có gồm năm danh mục sản phẩm. Laptop có doanh thu cao nhất, chiếm khoảng 20,16% tổng doanh thu theo danh mục; Tai nghe có tỷ trọng thấp nhất, khoảng 19,76%.

	Danh mục character varying (100)	Doanh thu numeric	Tỷ trọng (%) numeric
1	Laptop	28186637859163.34	20.16
2	Đồng hồ	28136644889324.62	20.13
3	Phụ kiện	27995923502293.04	20.03
4	Điện thoại	27836497333675.56	19.91
5	Tai nghe	27625540864058.28	19.76

Hình 6.2. Doanh thu theo danh mục

Tỷ trọng doanh thu giữa các danh mục tương đối đồng đều. Kết quả cho thấy không có danh mục nào chiếm ưu thế quá lớn về doanh thu trong bộ dữ liệu hiện tại. Tuy nhiên, doanh thu cao không đồng nghĩa với lợi nhuận cao do dự án chưa tính đến chi phí sản phẩm và các khoản giảm trừ.

6.3.3. Phân tích Top 10 sản phẩm

Kết quả xếp hạng cho thấy sản phẩm Product 3856, mã sản phẩm 3871, thuộc danh mục Phụ kiện, đứng đầu với doanh thu 17.865.651.523,46 và số lượng bán 767.

	Hạng bigint	Mã sản phẩm bigint	Tên sản phẩm character varying (200)	Danh mục character varying (100)	Doanh thu numeric	Số lượng bán bigint
1	1	3871	Product 3856	Phụ kiện	17865651523.46	767
2	2	9573	Product 9558	Đồng hồ	17539550974.14	753
3	3	6534	Product 6519	Phụ kiện	17423086492.24	748
4	4	7750	Product 7735	Laptop	17423086492.24	748
5	5	9316	Product 9301	Đồng hồ	17399793595.86	747
6	6	6333	Product 6318	Điện thoại	17399793595.86	747
7	7	8734	Product 8719	Đồng hồ	17399793595.86	747
8	8	788	Product 773	Điện thoại	17376500699.48	746
9	9	438	Product 423	Laptop	17353207803.10	745
10	10	8381	Product 8366	Điện thoại	17329914906.72	744

Hình 6.3 Top 10 sản phẩm bán chạy

Bảng xếp hạng giúp xác định những sản phẩm đóng góp doanh thu lớn nhất trong dữ liệu, đồng thời hỗ trợ so sánh giữa các sản phẩm và danh mục. Tên sản phẩm hiện được lưu theo dạng mã minh họa như Product 3856; vì vậy, kết quả chủ yếu phục vụ mục tiêu kiểm thử và phân tích kỹ thuật của dự án.

6.4. Đánh giá chung

Dự án đã xây dựng được quy trình phân tích dữ liệu từ Data Warehouse bằng Spark, tổng hợp doanh thu theo tháng, danh mục và sản phẩm, đồng thời đối chiếu kết quả theo tháng với PostgreSQL. Các tệp CSV đầu ra được sử dụng làm nguồn dữ liệu cho Dashboard Power BI.

Kết quả hiện tại ghi nhận 32 tháng dữ liệu, 1.000.000 đơn hàng và 6.001.039 sản phẩm bán ra. Quy trình giúp minh họa khả năng phối hợp giữa hệ quản trị cơ sở dữ liệu, công cụ xử lý dữ liệu và công cụ trực quan hóa.

Một số giới hạn cần lưu ý là dữ liệu chưa được lọc riêng theo trạng thái đơn hàng, đơn vị tiền tệ chưa được xác định rõ trong các tệp kết quả, và chưa có dữ liệu chi phí để tính lợi nhuận. Vì vậy, các nhận xét chỉ phản ánh kết quả tổng hợp trên dữ liệu hiện có.

7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1. Kết luận

Đề tài "Tối ưu và phân tích dữ liệu bán hàng quy mô lớn bằng SQL và các công nghệ Big Data" được thực hiện nhằm vận dụng kiến thức về cơ sở dữ liệu, tối ưu truy vấn, tích hợp dữ liệu và phân tích dữ liệu.

Trong quá trình thực hiện, nhóm đã xây dựng cơ sở dữ liệu bán hàng trên PostgreSQL với hơn 3,11 triệu bản ghi; triển khai các truy vấn SQL nâng cao bằng JOIN, CTE và Window Functions; sử dụng EXPLAIN ANALYZE để phân tích kế hoạch thực thi và thử nghiệm tối ưu bằng INDEX.

Bên cạnh đó, nhóm xây dựng Data Warehouse theo mô hình Star Schema và thực hiện đối chiếu dữ liệu sau ETL. Kết quả kiểm tra đã ghi nhận số lượng bản ghi và tổng doanh thu giữa nguồn với bảng Fact khớp nhau.

Đề tài giúp nhóm hiểu rõ hơn quy trình quản lý và khai thác dữ liệu quy mô lớn, đồng thời làm quen với các công nghệ PostgreSQL, Python, PySpark và Power BI. Tuy nhiên, các nội dung liên quan đến xử lý Spark và Dashboard cần được xác nhận bằng kết quả chạy thực tế để hoàn thiện sản phẩm.

7.2. Hướng phát triển

Trong tương lai, hệ thống có thể được mở rộng theo các hướng sau:

1. Tối ưu SQL chuyên sâu: Thử nghiệm chỉ mục kết hợp, tối ưu điều kiện truy vấn và so sánh nhiều phương án thực thi trên cùng bộ dữ liệu.
2. Tự động hóa ETL: Xây dựng quy trình nạp dữ liệu định kỳ, bổ sung ghi log, kiểm tra lỗi và cơ chế xử lý khi dữ liệu không hợp lệ.
3. Mở rộng phân tích bằng PySpark: Hoàn thiện môi trường Spark, thử nghiệm với bộ dữ liệu lớn hơn và so sánh thời gian xử lý với các truy vấn SQL tương ứng.
4. Hoàn thiện Dashboard: Bổ sung bộ lọc theo thời gian, danh mục và trạng thái; xây dựng các chỉ số doanh thu và số lượng đơn hàng có khả năng tương tác.
5. Nâng cao chất lượng dữ liệu: Xây dựng các quy tắc kiểm tra tự động để phát hiện dữ liệu thiếu, trùng lặp hoặc không nhất quán trước khi nạp vào Data Warehouse.
6. Đánh giá khả năng mở rộng: Thử nghiệm với dữ liệu có quy mô lớn hơn và các môi trường xử lý khác nhau để xác định giới hạn hiệu năng của hệ thống.

Những hướng phát triển trên sẽ giúp hệ thống hoàn thiện hơn, nâng cao khả năng tự động hóa và hỗ trợ phân tích dữ liệu hiệu quả hơn.

TÀI LIỆU THAM KHẢO

[1]. PostgreSQL Global Development Group. PostgreSQL Documentation. https://www.postgresql.org/docs/

[2]. Apache Software Foundation. Apache Spark Documentation. https://spark.apache.org/docs/latest/

[3]. Python Software Foundation. Python Documentation. https://docs.python.org/3/

[4]. Microsoft. Power BI Documentation. https://learn.microsoft.com/power-bi/

[5]. Coursera :

- Introduction to Big Data,
- Big Data Modeling and Management Systems,
- Big Data Integration and Processing.